

Conceptos básicos de programación

Nota: Esta sección está especialmente dirigida a estudiantes de **ASIR** que no han tenido módulos previos de programación. Los estudiantes de **DAW** pueden usar esta sección como repaso de conceptos fundamentales, vistos en el módulos de Programación.

¿Qué es la programación?

La **programación** es el proceso de crear instrucciones que una computadora puede entender y ejecutar para resolver problemas o realizar tareas específicas. Un **programa** es una **secuencia ordenada de instrucciones** que le dice al ordenador qué hacer y cómo hacerlo.

Imagina que estás explicándole a alguien cómo hacer una taza de café:

1. Llenar la cafetera con agua
2. Añadir café molido al filtro
3. Encender la cafetera
4. Esperar a que termine
5. Servir en una taza

De manera similar, programar es darle instrucciones paso a paso al ordenador.

Algoritmos y resolución de problemas

¿Qué es un algoritmo?

Un **algoritmo** es una secuencia finita de pasos bien definidos que resuelven un problema específico. Los algoritmos están por todas partes:

- Una receta de cocina
- Las instrucciones para montar un mueble
- Los pasos para calcular el área de un círculo
- La ruta para ir de casa al trabajo

Características de un buen algoritmo

1. **Finito:** Debe terminar en un número determinado de pasos
2. **Preciso:** Cada paso debe estar claramente definido

3. **Entrada:** Debe tener cero o más datos de entrada
4. **Salida:** Debe producir al menos un resultado
5. **Efectivo:** Cada paso debe ser realizable

Los algoritmos pueden describirse (reflejarse o visualizarse) utilizando diferentes **técnicas descriptivas**, como el **pseudocódigo** o los diagramas de flujo.

Pseudocódigo

El **pseudocódigo** es una forma de escribir algoritmos usando un lenguaje entre el humano y el de programación. Nos ayuda a pensar la lógica antes de escribir código real.

Ejemplo: Algoritmo para encontrar el mayor de dos números

```
ALGORITMO Mayor_de_dos
INICIO
    ESCRIBIR "Introduce el primer número:"
    LEER numero1
    ESCRIBIR "Introduce el segundo número:"
    LEER numero2

    SI numero1 > numero2 ENTONCES
        ESCRIBIR "El mayor es:", numero1
    SINO
        ESCRIBIR "El mayor es:", numero2
    FIN_SI
FIN
```



Elementos básicos de programación

Todos los lenguajes de programación comparten una serie de elementos comunes a todos ellos, y comunes también a muchos de los elementos de cualquier lenguaje o idioma que entendamos los humanos. En concreto, un lenguaje de programación consta de:

- Identificadores
- Palabras reservadas
- Variables
- Constantes
- Literales
- Operadores

- Comentarios
- Delimitadores

Palabras reservadas

En un lenguaje de programación, las **palabras reservadas** son términos que tienen un significado especial y no pueden usarse como nombres de variables, funciones u otros identificadores. Estas palabras forman parte de la sintaxis del lenguaje y son esenciales para escribir código correctamente.

Ejemplos de palabras reservadas en Javascript:

- `if`, `else` para estructuras condicionales
- `for`, `while` para bucles
- `function` para definir funciones
- `return` para devolver valores desde funciones
- `var`, `let`, `const` para declarar variables
- `try`, `catch` para manejo de excepciones

Todas estas palabras reservadas las iremos entendiendo en sucesivos apartados.

Variables

Una **variable** es un espacio en la memoria del ordenador donde podemos almacenar un valor que puede cambiar durante la ejecución del programa.

Piensa en una variable como una caja con una etiqueta:

- La **etiqueta** es el nombre de la variable
- El **contenido** es el valor que almacena
- Podemos **cambiar** el contenido cuando queramos

Reglas para nombrar variables:

Aunque pueden variar entre lenguajes, las más comunes son:

- No pueden empezar por números
- No pueden contener espacios
- No pueden usar **palabras reservadas** del lenguaje
- Es recomendable usar nombres descriptivos

A continuación se muestra un ejemplo de declaración de variables en Javascript, utilizando la palabra reservada `var`:

```
// Ejemplos de nombres de variables
var edad; // ✅ Correcto
var nombreUsuario; // ✅ Correcto
var 2nombre; // ❌ Incorrecto (empieza por número)
var nombre usuario; // ❌ Incorrecto (contiene espacio)
```



Tipos de datos básicos

Los **tipos de datos** definen qué clase de información puede almacenar una variable:

1. Números (Number)

Para almacenar valores numéricos:

```
var edad = 25;
var precio = 19.99;
var temperatura = -5;
```



2. Texto/Cadenas (String)

Sirven para almacenar texto:

```
var nombre = "Ana";
var mensaje = "Hola mundo";
var direccion = "Calle Mayor, 123";
```



3. Booleanos (Boolean)

Para almacenar valores de **verdadero** o **falso**:

```
var esMayorDeEdad = true;
var tieneDescuento = false;
```



Constantes

Las **constantes** son valores que no cambian durante la ejecución del programa:

```
const PI = 3.14159;
const NOMBRE_EMPRESA = "Mi Empresa S.L.";
```



Literales

Los **literales** son los valores fijos que asignamos a las variables o usamos directamente en el código. Pueden ser de diferentes tipos:

- **Números:** 42, 3.14, -5
- **Cadenas de texto:** "Hola", 'Mundo', `Hola \${nombre}`
- **Booleanos:** true, false
- **Nulos:** null
- **Indefinidos:** undefined

Diferencia entre variables, constantes y literales

- **Variable:** Espacio en memoria que puede cambiar (ej. `var edad = 25;`)
- **Constante:** Espacio en memoria que no cambia (ej. `const PI = 3.14;`)
- **Literal:** Valor fijo usado en el código (ej. 25, "Hola"). No ocupa espacio en memoria, es un valor al que nos referimos directamente por dicho valor.

Comentarios

Los **comentarios** son texto que el programador escribe para explicar el código pero que el ordenador ignora:

```
// Esto es un comentario de una línea
```



```
/*
```

```
Esto es un comentario  
de varias líneas
```

```
*/
```

```
var edad = 18; // También puedo comentar al final de una línea
```

Delimitadores

Los **delimitadores** son símbolos que usamos para estructurar el código y separar diferentes partes. Algunos ejemplos comunes son:

- Llaves `{ }`: Para agrupar bloques de código (funciones, condicionales, bucles)
- Paréntesis `( )`: Para agrupar condiciones o parámetros de funciones
- Corchetes `[ ]`: Para definir arrays o listas
- Punto y coma `;`: Para indicar el final de una instrucción (opcional en JavaScript, pero recomendable)
- Comillas `' '` o `" "`: Para definir cadenas de texto

Cada delimitador tiene un propósito específico y es fundamental para que el código sea correcto y funcione como se espera. Por tanto, no son intercambiables y, en la mayoría de casos, no se pueden omitir.

Operadores

Los **operadores** son símbolos que nos permiten realizar operaciones con los datos:

Operadores aritméticos

```
var a = 10;
var b = 3;

var suma = a + b;           // 13
var resta = a - b;          // 7
var multiplicacion = a * b; // 30
var division = a / b;       // 3.333...
var resto = a % b;          // 1 (resto de la división)
```



Operadores de comparación

Devuelven `true` o `false`:

```
var x = 5;
var y = 8;

x == y; // false (igual a)
x != y; // true (distinto de)
x < y;  // true (menor que)
x > y;  // false (mayor que)
```



```
x <= y; // true (menor o igual que)
x >= y; // false (mayor o igual que)
```

Operadores lógicos

```
var tieneDinero = true;
var esFinDeSemana = false;

// Y lógico (AND) - ambos deben ser true
var puedeComprar = tieneDinero && esFinDeSemana; // false

// O lógico (OR) - al menos uno debe ser true
var puedeDescansar = tieneDinero || esFinDeSemana; // true

// NO lógico (NOT) - invierte el valor
var noTieneDinero = !tieneDinero; // false
```



Estructuras de control

Las **estructuras de control** nos permiten decidir qué código ejecutar y cuándo ejecutarlo. Distinguimos principalmente:

- Estructuras condicionales
- Estructuras repetitivas (bucles)

Estructuras condicionales

Las **estructuras condicionales** nos permiten ejecutar código solo si se cumple una condición. En JavaScript, las principales estructuras condicionales son:

if (si)

```
var edad = 18;

if (edad >= 18) {
  console.log("Eres mayor de edad");
}
```



if-else (si-sino)

```
var nota = 7;

if (nota >= 5) {
    console.log("Has aprobado");
} else {
    console.log("Has suspendido");
}
```



if-else if-else (múltiples condiciones)

```
var nota = 8;

if (nota >= 9) {
    console.log("Sobresaliente");
} else if (nota >= 7) {
    console.log("Notable");
} else if (nota >= 5) {
    console.log("Aprobado");
} else {
    console.log("Suspenso");
}
```



Estructuras repetitivas (bucles)

Los **bucles** nos permiten repetir código varias veces.

Bucle for

Cuando sabemos cuántas veces queremos repetir:

```
// Contar del 1 al 5
for (var i = 1; i <= 5; i++) {
    console.log("Número: " + i);
}
```



Bucle while

Mientras se cumpla una condición:




```
var contador = 0;

while (contador < 3) {
  console.log("Contador: " + contador);
  contador++; // Incrementar contador
}
```

Cuándo usar cada bucle

- **for**: Cuando sabes cuántas repeticiones necesitas. Podemos saberlo en tiempo de programación (usando un literal) pero también en tiempo de ejecución (por ejemplo, recorriendo los elementos de un array, los caracteres de una cadena, etc.)
- **while**: Cuando dependes de una condición que puede cambiar (por ejemplo, leer datos hasta que el usuario decida parar).

Funciones

Una **función** es un bloque de código que realiza una tarea específica y que podemos usar (llamar) cuando la necesitemos.

¿Por qué usar funciones?

1. **Reutilización**: Evitamos repetir código
2. **Organización**: El código es más ordenado
3. **Mantenimiento**: Es más fácil hacer cambios
4. **Legibilidad**: Es más fácil entender qué hace cada parte

Crear una función

```
function saludar() {
  console.log("¡Hola!");
}

// Llamar (usar) la función
saludar(); // Muestra: ¡Hola!
```



Funciones con parámetros

Los **parámetros** son datos que le pasamos a la función:

```
function saludarPersona(nombre) {  
    console.log("¡Hola, " + nombre + "!");  
}  
  
saludarPersona("Ana");    // Muestra: ¡Hola, Ana!  
saludarPersona("Carlos"); // Muestra: ¡Hola, Carlos!
```



Funciones que devuelven valores

```
function sumar(a, b) {  
    var resultado = a + b;  
    return resultado; // Devolver el resultado  
}  
  
var total = sumar(5, 3); // total = 8  
console.log(total);
```



Ámbito de las variables (scope)

Las variables tienen un **ámbito** que determina dónde pueden ser usadas:

```
var variableGlobal = "Soy global";  
  
function miFuncion() {  
    var variableLocal = "Soy local";  
    console.log(variableGlobal); // ✅ Funciona  
    console.log(variableLocal);  // ✅ Funciona  
}  
  
console.log(variableGlobal); // ✅ Funciona  
console.log(variableLocal);  // ❌ Error - no existe aquí
```



Estructuras de datos básicas

Arrays (Arreglos/Listas)

Un **array** es una lista ordenada de elementos:



```
var frutas = ["manzana", "banana", "naranja"];
var numeros = [1, 2, 3, 4, 5];

// Acceder a elementos (empezando desde 0)
console.log(frutas[0]); // "manzana"
console.log(frutas[1]); // "banana"

// Añadir elementos
frutas.push("kiwi"); // Añade al final

// Saber cuántos elementos hay
console.log(frutas.length); // 4
```

Objetos

Un **objeto** es una colección de propiedades (características):

```
var persona = {
  nombre: "Ana",
  edad: 25,
  esEstudiante: true
};

// Acceder a propiedades
console.log(persona.nombre); // "Ana"
console.log(persona["edad"]); // 25

// Modificar propiedades
persona.edad = 26;
```



Ejemplo práctico: Calculadora simple

Vamos a crear una función que simule una calculadora:

```
function calculadora(numero1, numero2, operacion) {
  var resultado;

  if (operacion === "suma") {
    resultado = numero1 + numero2;
  } else if (operacion === "resta") {
    resultado = numero1 - numero2;
  }
}
```



```

    } else if (operacion === "multiplicacion") {
        resultado = numero1 * numero2;
    } else if (operacion === "division") {
        if (numero2 !== 0) {
            resultado = numero1 / numero2;
        } else {
            return "Error: No se puede dividir por cero";
        }
    } else {
        return "Error: Operación no válida";
    }

    return resultado;
}

// Usar la calculadora
console.log(calculadora(10, 5, "suma"));           // 15
console.log(calculadora(10, 3, "multiplicacion")); // 30
console.log(calculadora(10, 0, "division"));       // Error: No se puede dividir

```

Consejos para principiantes

1. Practica regularmente

- La programación se aprende practicando, no solo leyendo
- Intenta resolver pequeños problemas cada día

2. Usa nombres descriptivos

```

// ❌ Mal
var x = 18;
var y = x * 2;

// ✅ Bien
var edad = 18;
var edadEnMeses = edad * 12;

```



3. Comenta tu código

```

// Calcular el área de un rectángulo
var base = 10;

```



```
var altura = 5;  
var area = base * altura; // Fórmula: base × altura
```

4. Divide problemas complejos

- Si un problema es muy grande, divídelo en partes más pequeñas
- Resuelve cada parte por separado
- Luego únelas para formar la solución completa

5. No te desanimes con los errores

- Los errores son normales y parte del aprendizaje
- Cada error te enseña algo nuevo
- Usa la consola del navegador para ver los errores

Próximos pasos

Ahora que conoces los conceptos básicos de programación, estás preparado para:

1. **Aprender JavaScript específicamente** para programación web
2. **Entender cómo funciona el DOM** (estructura de las páginas web)
3. **Crear interactividad** en páginas web
4. **Manejar eventos** como clics y teclado
5. **Desarrollar proyectos reales**

Recuerda que estos conceptos son la base de toda programación, no solo de JavaScript. Una vez que los dominas, te será mucho más fácil aprender cualquier otro lenguaje de programación.